M&I MINI PROJECT- Water Quality Monitoring Using a TDS Sensor

BT23ECE001, BT23ECE003, BT23ECE007, BT23ECE011

Introduction:

Water quality is a critical parameter for various applications, ranging from drinking water standards to agricultural and industrial processes. One of the key indicators of water quality is the Total Dissolved Solids (TDS), which represents the total concentration of dissolved substances such as salts, minerals, and metals in water. A high TDS value can indicate the presence of impurities, which may affect the taste, safety, and usability of the water.

The TDS Sensor project is designed to accurately measure the TDS levels in a water sample, providing a quantitative assessment of water quality. The TDS value is expressed in parts per million (ppm), representing the weight of dissolved solids in a given volume of water. For example, a reading of 500 ppm means that there are 500 milligrams of dissolved solids per liter of water. This project is particularly relevant for monitoring drinking water quality, assessing the health of aquatic ecosystems, and evaluating water used in industrial applications.

The key component of this project is a TDS sensor, which operates based on the principle of electrical conductivity. Pure water has low conductivity due to the absence of charged particles, but the presence of dissolved substances increases conductivity. By measuring the electrical conductivity of the water sample, the TDS sensor can infer the concentration of dissolved solids. The sensor's output, an analog voltage signal proportional to the TDS level, is processed by a microcontroller, typically an Arduino, to calculate the precise TDS value.

This project offers several advantages:

- **Cost-effectiveness: The TDS sensor provides a reliable way to assess water quality without the need for expensive laboratory tests.**
- **Real-time Monitoring: With an LCD display, TDS levels can be monitored instantly, allowing for quick decision-making.**
- **Ease of Use: The setup is simple and portable, making it suitable for fieldwork, research, and educational purposes.**

Components Used and Their Specifications:

1. **TDS Sensor**
   - **Range**: 0 - 1000 ppm (parts per million)

    - **Output Type**: Analog Voltage
    - **Accuracy**: ±10% F.S. (Full Scale)
    - **Working Voltage**: 3.3V to 5.5V
    - **Temperature Compensation**: Yes
2. **Arduino Uno**
    - **Microcontroller**: ATmega328P
    - **Operating Voltage**: 5V
    - **Analog I/O Pins**: 6 (used for analog input from TDS sensor)
    - **Clock Speed**: 16 MHz
3. **LCD Display with I2C Module**
    - **Display Type**: 16x2 LCD
    - **Interface**: I2C (saves pin usage)
    - **Operating Voltage**: 5V
    - **Backlight**: Yes (adjustable)
4. **Connecting Wires**: Standard jumper wires for connecting the circuit.
5. **Breadboard**: For easy circuit assembly and testing.
6. **Power Bank**: Gives constant 5v and can adjust mA as per requirement(2.4A Max)

Working Principle:

The TDS sensor measures the Total Dissolved Solids in water by assessing its electrical conductivity, as dissolved solids like salts, minerals, and metals increase conductivity. Here's how the process unfolds step-by-step:

1. **Submerging the Probes**: The TDS sensor has two metal probes that are submerged in the water sample. These probes act as electrodes, creating an electrical circuit within the liquid.
2. **Conductivity Measurement**: When an electrical current is applied to the probes, ions present in the dissolved solids facilitate the flow of current. A pure water sample, with low ion content, exhibits minimal conductivity, whereas water with a higher concentration of dissolved substances shows greater conductivity.
3. **Analog Signal Generation**: The sensor measures this conductivity and converts it into an analog voltage signal. The higher the conductivity, the greater the voltage produced by the sensor.
4. **Analog-to-Digital Conversion**: The analog signal is fed into the Arduino's analog input pins. The Arduino, using its Analog-to-Digital Converter (ADC), transforms this voltage into a digital value. This digital value is proportional to the concentration of dissolved solids in the water.
5. **TDS Calculation**: A calibration formula, specific to the sensor model, is applied within the Arduino code to convert the digital reading into a TDS value in ppm. This formula factors in the sensor’s characteristics and may include a multiplier or an offset to account for accuracy.
6. **Display Output**: Finally, the calculated TDS value is sent to the 16x2 LCD display via the I2C interface, providing a clear and real-time reading of the water's dissolved solids. This allows users to immediately interpret the water quality and make informed decisions.

Uploaded code on Arduino:

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>


#define TdsSensorPin A0   // Pin connected to the TDS sensor

#define VREF 5.0        // Analog reference voltage (assuming 5V Arduino)

#define SCOUNT 30        // Sampling times


int analogBuffer[SCOUNT];   // Store analog values

int analogBufferTemp[SCOUNT];

int analogBufferIndex = 0;

float averageVoltage = 0;

float tdsValue = 0;

float temperature = 25;

float x = 0;  // Set default temperature for compensation
LiquidCrystal_I2C lcd(0x27, 16, 2);  // Set LCD address to 0x27
void setup() {

 lcd.init();

 lcd.backlight();

 lcd.setCursor(0, 0);

 lcd.print("TDS Meter");

 delay(2000);

 Serial.begin(9600);

}
void loop() {

 static unsigned long analogSampleTimepoint = millis();

 if (millis() - analogSampleTimepoint > 40U) {

  analogSampleTimepoint = millis();
```

```
  analogBuffer[analogBufferIndex] = analogRead(TdsSensorPin);
  analogBufferIndex++;
  if (analogBufferIndex == SCOUNT) {
    analogBufferIndex = 0;
  }
}
static unsigned long printTimepoint = millis();
if (millis() - printTimepoint > 800U) {
  printTimepoint = millis();
  for (int i = 0; i < SCOUNT; i++) {f
    analogBufferTemp[i] = analogBuffer[i];
  }
  averageVoltage = getMedianNum(analogBufferTemp, SCOUNT) * (float)VREF / 1024.0;
  // TDS calculation (based on voltage)
  float compensationCoefficient = 1.0 + 0.02 * (temperature - 25.0);
  float compensationVoltage = averageVoltage / compensationCoefficient;
  tdsValue = 1/ (133.42 * compensationVoltage * compensationVoltage * compensationVoltage
          - 255.86 * compensationVoltage * compensationVoltage
          + 857.39 * compensationVoltage) * 0.5;  // Calibration coefficient for accuracy
  x = 2642.86*tdsValue + 123.07;
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("TDS: ");
  lcd.print(x, 5);
  lcd.print(" ppm");

  Serial.print("TDS Value: ");
  Serial.print(tdsValue, 0);
```

```
    Serial.println(" ppm");
  }
}
// Helper function for median filtering
int getMedianNum(int bArray[], int iFilterLen) {
  int bTab[iFilterLen];
  for (int i = 0; i < iFilterLen; i++) {
    bTab[i] = bArray[i];
  }
  int i, j, bTemp;
  for (j = 0; j < iFilterLen - 1; j++) {
    for (i = 0; i < iFilterLen - j - 1; i++) {
      if (bTab[i] > bTab[i + 1]) {
        bTemp = bTab[i];
        bTab[i] = bTab[i + 1];
        bTab[i + 1] = bTemp;
      }
    }
  }
  if ((iFilterLen & 1) > 0) {
    bTemp = bTab[(iFilterLen - 1) / 2];
  } else {
    bTemp = (bTab[iFilterLen / 2] + bTab[iFilterLen / 2 - 1]) / 2;
  }
  return bTemp;
}
```